Практическая работа №4. Чтение данных из SQL Server с помощью SqlDataReader

Цель работы

Научиться получать данные из SQL Server с помощью `SqlDataReader`, последовательно читать строки результата, работать с типами данных C# и обрабатывать пустые результаты запроса. Используется база данных `coffee_shop`.

Задание

Часть 1. Подготовка проекта

Создайте консольное приложение C# или продолжите работу с проектом предыдущей практики. Убедитесь, что установлен пакет `Microsoft.Data.SqlClient`.

Подготовьте строку подключения:

```
string connectionString =
    "Server=localhost;Database=coffee_shop;Trusted_Connection
=True;TrustServerCertificate=True;";
```

При необходимости замените имя сервера и параметры подключения на свои.

Часть 2. Чтение списка товаров

Получите список товаров из таблицы `products`. Выведите идентификатор, название, цену и доступность. Используйте `ExecuteReader()` и `Read()`.

```sql
SELECT id, name, price, is_available
FROM products;
```

```
string sql = @"
    SELECT id, name, price, is_available
    FROM products";

using SqlConnection connection = new SqlConnection(connection
String);
connection.Open();

using SqlCommand command = new SqlCommand(sql, connection);
using SqlDataReader reader = command.ExecuteReader();

while (reader.Read())
{
    int id = reader.GetInt32(0);
    string name = reader.GetString(1);
    decimal price = reader.GetDecimal(2);
    bool isAvailable = reader.GetBoolean(3);

    Console.WriteLine($"{id}. {name}");
    Console.WriteLine($"Цена: {price} руб.");
    Console.WriteLine($"Доступен: {(isAvailable ? "Да" : "Не
т")}");
    Console.WriteLine();
}
```

Порядок столбцов в запросе должен соответствовать индексам, используемым в методах чтения.

Часть 3. Проверка наличия данных

Доработайте программу: перед чтением проверьте `HasRows`. Если запрос не вернул товаров, выведите `Товары не найдены.`. В противном случае выведите

список.

Часть 4. Поиск товара по идентификатору

Пользователь вводит `id`, программа выводит название, цену и доступность найденного товара. Используйте параметризованный запрос:

```
SELECT id, name, price, is_available
FROM products
WHERE id = @id;
```

Передайте идентификатор через параметр `@id`. Если товар не найден, сообщите об этом пользователю. Поскольку идентификатор уникален, результат содержит не более одной строки.

Часть 5. Поиск товаров по названию

Добавьте поиск товаров по части названия.

```
SELECT id, name, price
FROM products
WHERE name LIKE @search;
```

Передайте параметр с символами `%` в начале и конце. Например, поиск по слову `кофе` должен выводить товары, в названии которых встречается эта последовательность. Выведите идентификатор, название и цену каждого совпадения. Если совпадений нет, выведите сообщение.

Часть 6. Чтение данных о клиентах

Выведите список клиентов из таблицы `customers`: `id`, `full_name`, `phone`, `email`.

```
SELECT id, full_name, phone, email
FROM customers;
```

Учитывайте возможность `NULL`. Например, для столбца `email`:

```
string email = reader.IsDBNull(3)
    ? "Не указан"
    : reader.GetString(3);
```

Даже если в текущих данных все адреса заполнены, предусмотрите такую проверку.

Часть 7. Формирование отчёта по заказам

Выведите список заказов из таблицы `orders`. Для каждого заказа отобразите идентификатор, дату оформления, статус и идентификатор клиента.

```
SELECT id, customer_id, order_date, status
FROM orders;
```

Для даты используйте `GetDateTime()`, для остальных полей — методы, соответствующие типам данных. Дату можно вывести так:

```
Console.WriteLine(orderDate.ToString("dd.MM.yyyy HH:mm"));
```

Часть 8. Самостоятельная часть

Добавьте следующие возможности:

1. Вывести доступные товары, отсортированные по цене от меньшей к большей.
2. Найти товары дороже значения, введённого пользователем.
3. Найти клиента по части Ф. И. О.
4. Вывести заказы с заданным пользователем статусом.
5. Для каждого запроса предусмотреть сообщение, если записи отсутствуют.

Все пользовательские значения передавайте через параметры. Для чтения результатов используйте `SqlDataReader`. По возможности организуйте операции через меню.

Что необходимо продемонстрировать преподавателю

- Подключение к базе `coffee_shop`.
- Чтение нескольких строк с помощью `SqlDataReader`.
- Использование `Read()` и типизированных методов.
- Проверку наличия результатов.
- Поиск по идентификатору и части названия.
- Обработку возможного `NULL`.
- Использование параметризованных запросов.
- Освобождение ресурсов через `using`.

Выводы

Ответьте письменно:

1. Для чего используется `SqlDataReader`?
2. Как выполняется последовательное чтение строк?
3. Почему важно правильно выбирать методы чтения данных?
4. Как обрабатываются `NULL` и пустые результаты?
5. Почему пользовательские значения нужно передавать через параметры?

Дополнительное задание

Добавьте вывод списка заказов с именами клиентов. Соедините таблицы `orders` и `customers` через `JOIN`. Выведите идентификатор заказа, Ф. И. О. клиента, дату и статус заказа. Полученные данные прочитайте с помощью `SqlDataReader`.